BERT: the Stage 1-8 experiment path

Stage-by-stage comparisons and the selected BERT run. Scores are decimals on the shared validation set.

BASE CONFIGURATION - RUN 01 / bert-base-uncased

LR	Batch	Epochs	Dropout	Decay	Optimizer	Seed	Frozen blocks	Early stopping
3e-5	16	3	0.10	0.01	AdamW	42	0	Off

Stage / new runs	Run-by-run comparison	Decision and observed result
Stage 1 - Epochs Runs 01-03	**Run 01:** maximum 3 epochs. **Run 02:** maximum 4 epochs. **Run 03:** maximum 5 epochs.	**Decision:** keep 5 epochs (03). **Negative F1:** 0.8393. **Purpose:** choose a training budget.
Stage 2 - LR Runs 04-05	**Run 03 (reference):** LR 3e-5. **Run 04:** LR 2e-5. **Run 05:** LR 5e-5.	**Decision:** keep LR 3e-5 (03). **Negative F1, 03 / 04 / 05:** 0.8393 / 0.8283 / 0.8306.
Stage 3 - Batch Run 06	**Run 03 (reference):** batch 16. **Run 06:** batch 32.	**Decision:** use batch 32 (06). **Negative F1:** 0.8435.
Stage 4 - Dropout Run 07	**Run 06 (reference):** dropout 0.10. **Run 07:** dropout 0.30.	**Decision:** use dropout .30 (07). **Negative F1:** 0.8457.
Stage 5 - Decay Run 08	**Run 07 (reference):** weight decay 0.01. **Run 08:** weight decay 0.05.	**Decision:** use decay .05 (08). **Negative F1:** 0.8500.
Stage 6 - Strategy Runs 09-10	**Run 08 (reference):** max 5 epochs; no freezing. **Run 09:** freeze first 6 encoder blocks. **Run 10:** max 8 epochs + early stopping.	**Decision:** select Run 10. **Negative F1, 08 / 09 / 10:** 0.8500 / 0.8554 / 0.8571. **Checkpoint:** epoch 5.
Stage 7 - Seeds Runs 11-12	**Run 10 (reference):** seed 42. **Run 11:** seed 43; same settings. **Run 12:** seed 44; same settings.	**Action:** assess stability. **Mean Negative F1:** 0.8479. **Sample SD:** 0.0080. **Best single seed:** 42.
Stage 8 - Hybrid Runs 13-20	**Grid - batch 32, weight decay .05** **Run 13:** LR 2.5e-5; dropout .20. **Run 14:** LR 2.5e-5; dropout .40. **Run 15:** LR 3.5e-5; dropout .20. **Run 16:** LR 3.5e-5; dropout .40. **Random - LR / batch / dropout / decay** **Run 17:** 2.935e-5 / 32 / .274 / .041. **Run 18:** 3.581e-5 / 16 / .381 / .031. **Run 19:** 3.306e-5 / 16 / .260 / .078. **Run 20:** 3.840e-5 / 32 / .327 / .065.	**Decision:** retain Run 10. **Outcome:** none of the 8 candidates beats it. **Best new candidate:** Run 17. **Negative F1:** 0.8515. **Common settings:** max 8 epochs, early stopping, seed 42, no frozen blocks. **Random-plan seed:** 2026.

RUN 10 - BEST OBSERVED BERT RUN

Negative F1 0.8571 | Macro F1 0.9170 | Accuracy 0.9601

LR	Batch	Dropout	Decay	Optimizer	Seed	Frozen blocks	Max epochs	Stop patience
3e-5	32	0.30	0.05	AdamW	42	0	8	2

Checkpoint: training stops after epoch 7; predictions use the restored epoch-5 model.

1. Highest Negative F1, Macro F1 and Accuracy among the supplied BERT runs.

2. Negative F1 improves by **0.0332**; validation errors fall from **53 to 43** versus Run 01.

3. Remains the best observed run after **eight additional hybrid-search candidates**.

Our approach and key findings

Why we tuned, how we searched, how we chose the model, and what the experiments showed.

Main question	Explanation and conclusion
Why tune BERT at all?	**Pretraining does not choose the best settings for our review dataset.** Learning rate, training length, batch size and regularization affect fine-tuning. We use validation results to find suitable settings, balancing learning the reviews against generalizing beyond the training data.
Why use sequential tuning first?	**Make the decision path clear and manageable.** We compare one main setting at a time, keep the current choice, and then move to the next setting. This helps us see which options look promising without testing every combination. It is an understandable first pass, but a setting that works well now may behave differently when other settings change.
Why add a hybrid search?	**Check combinations the first pass may miss.** Stage 8 tests four grid combinations and four random configurations nearby. The root file **stage8_search_plan.csv** records these eight candidates; each Stage-8 notebook loads the same plan. Keeping this file makes the exact search easy to repeat. Eight candidates still do not prove the settings are optimal.
How did we choose and use the model?	**Use Negative F1 on validation data, not test predictions.** Positive reviews are the majority, so Accuracy alone can hide weak Negative detection. Each run saves its best validation Negative-F1 checkpoint. We compare those results, check the selected setup with three seeds, and use the saved model to predict the unlabeled test reviews.
What did the full search show?	**The sequential search found Run 10, and Stage 8 did not replace it.** Run 10 has Negative F1 **0.8571**; the best new hybrid candidate reaches **0.8515**. We therefore retain Run 10 as the best observed BERT run. The hybrid search still adds useful evidence by testing alternatives, even though none improves the result.
What did we learn about training?	**More training or stronger regularization is not automatically better.** Higher dropout and frozen layers help catch Negative reviews, but may create more errors on Positive reviews. Run 10 offers the best balance under our selection metric. Its best checkpoint is epoch 5, before training stops at epoch 7, showing why checkpoint selection matters.
How strong is the conclusion?	**Run 10 is the observed winner, not a guaranteed winner every time.** The same settings across three seeds give mean Negative F1 **0.8479** and sample standard deviation **0.0080**. Some tuning gains are smaller than this variation. Report the best run and seed summary together; all choices reuse one validation set, and the test file has no ground-truth labels.
What is the story for the group report?	**We first built a clear tuning path, then checked combinations and repeatability.** Use the Stage 1-8 summary, Run-10 settings and results, and the seed caveat. Compare BERT with RNN and other models on the same split and metrics; do not claim our search strategy always wins.

How this supports the NTU report

Question summaries with the original course marks in brackets. BERT is one model alongside RNN and others.

NTU question	BERT evidence or discussion to include
(a) Which feature format did you choose, and how did you preprocess raw reviews into model inputs? **(10%)**	Uncased BERT tokenizer; truncation at 128 tokens; attention masks and dynamic padding. Describe the shared data split once for all models.
(b) Which classifier did you build, how is it structured and trained, and how can others reproduce the results? **(20%)**	Provide a small architecture figure: tokens [B,L] -> 12-layer BERT -> [B,768] -> 2 logits. State cross-entropy, AdamW and checkpoint selection; report three-seed mean/SD and supply code/run instructions.
(c) How did you determine the model parameters and settings, and why did you choose them? **(10%)**	Use a shortened Stage 1-8 table, the final configuration and selected findings from page 2. Explain Negative F1, sequential carry-forward, seed checks and the hybrid search outcome.
(d) What predictions does the selected classifier make on the test set, as recorded in submission.csv? **(20%)**	Submit test predictions for the final group model, selected using comparable validation results. **The Run-10 BERT prediction file has been verified and is listed below.** Use it if the group selects Run 10; test Accuracy/F1 cannot be measured without labels.
(e) What do one or two correctly and incorrectly classified test examples reveal about model strengths and weaknesses? **(10%)**	Ground-truth test labels are unavailable. Clearly labeled validation examples can support discussion, but must not be presented as verified test correctness. Review mixed sentiment and possible annotation mismatches.
(f) How do different feature representations affect resource consumption and classification accuracy? **(10%)**	Compare BERT and RNN/other feature choices using the same split and metrics. Report measured training time with caveats; no peak-memory or inference-latency benchmark was supplied.
(g) How would the method perform on hotel reviews without rating labels, and how could it be adapted? **(10%)**	Discuss domain shift and possible adaptation in the shared report. Product-review validation results do not establish hotel-review performance; no hotel experiment was run here.
(h) How would noisy hotel ratings affect the method, and which three approaches could improve it? **(10%)**	Discuss three proposed approaches, such as trusted-subset relabeling, noise-robust loss/reweighting, and out-of-fold filtering or soft labels. State that these are proposals, not tested BERT findings.

Run 10: verified output	File location and handoff
Project file path	**03_BERT_results/ BERT_Stage6_Freezing_and_Early_Stopping/ Run_10_EarlyStopping/submission.csv**
Checks passed	**1,851 predictions:** 254 Negative (0), 1,597 Positive (1). Columns: **reviews, predictions**. All rows match the processed test file in the same order.
What to submit	If the group chooses **BERT Run 10**, submit its **submission.csv** together with the **final report**.

Sources: NTU project brief (pages 2-3); split JSONs; stage8_search_plan.csv; run configurations, metrics, training logs and prediction CSVs. Questions are paraphrased; course marks are unchanged.

Supplementary test evaluation

Using GPT-OSS 120B pseudo-labels to evaluate the saved BERT Run 10 predictions.

What we did

1 **Validate the labeler**

GPT-OSS 120B labeled 1,078 validation reviews without seeing their true labels. We then checked its predictions against the existing labels.

2 **Label the test reviews**

We used the same model and prompt on all 1,851 unlabeled test reviews, one review per request. It assigned 309 Negative and 1,542 Positive labels.

3 **Compare the saved predictions**

We matched those labels to the saved BERT Run 10 predictions after checking that all review texts and row order agreed. No retraining was needed.

Results

Comparison	Reviews	Accuracy	Negative F1
GPT-OSS vs. true validation labels	1,078	0.9601	0.8732
BERT Run 10 vs. GPT-OSS test labels	1,851	0.9530	0.8455

Run 10 matched 1,764 of 1,851 pseudo-labels. Of the 87 disagreements, 71 were Negative according to GPT-OSS but Positive according to BERT.

How to read the test score

The validation check supports using GPT-OSS as a reference for this extra experiment. The test accuracy above is measured against model-generated labels, not human ground truth. The official submission remains the selected model's prediction CSV.

Evidence: BERT_Extra_GPT_OSS_Evaluation/validation/validation_summary.txt and BERT_Extra_GPT_OSS_Evaluation/test/test_evaluation.txt.